

JoBot Unified Master Plan

From Feature-Complete Prototype to Production-Grade
Autonomous Job-Application Operating System

Version 2.0 · 2026-08-15

Status: AUTHORITATIVE — PENDING EXECUTION

Repository: <https://github.com/aryansinghnagar/JoBot>

Synthesised from Plan1–Plan6, Plan1.pdf, live repository audit,
and AGENTS.md doctrine. All essential ideas retained and de-conflicted.

Core Production Invariant

NO ACTION WITHOUT A STATE · NO STATE WITHOUT AN EVENT

NO COMPLETION WITHOUT VERIFICATION · NO SIDE EFFECT WITHOUT POLICY

NO RETRY WITHOUT IDEMPOTENCY · NO LONG RUN WITHOUT CHECKPOINT

NO MEMORY WITHOUT PROVENANCE · NO AUTONOMY WITHOUT MEASUREMENT

From Feature-Complete Prototype to Production-Grade Autonomous Job-Application Operating System

Version: 2.0 · 2026-08-15

Status: AUTHORITATIVE — PENDING EXECUTION

Repository: <https://github.com/aryansinghnagar/JoBot>

Target: v1.0.0 production-ready, releasable, commercially usable open-source product

Guiding Doctrine: AGENTS.md (zero fabrication, verification-first, file-based state, one-change eval loops, local-first, human-governed autonomy, observable over clever)

Executive Summary

JoBot already possesses the architectural chassis of an ambitious local-first agent platform:

- 12-phase Application Submission Pipeline (ASP) with saga orchestration and Definition-of-Done gates
- Honest per-site adapters (Greenhouse, Lever, Ashby, SmartRecruiters APIs; Naukri / LinkedIn / Workday browser paths; 8 JobSpy boards)
- Provider-neutral ModelRouter (12 providers, cost-aware fallback)
- SQLite WAL + Fernet vault + OS keyring
- Tauri 2 + React GUI shell + 22-method JSON-RPC sidecar
- Policy engine, circuit breakers, quarantine, traces, doctor, backup/migrate
- 359 hermetic pytest + 18 vitest, multi-OS CI, SBOM + provenance, CodeQL, Dependabot

The central recommendation (shared by every source plan) is therefore not to rewrite and not to immediately add volume-oriented features.

Make one end-to-end job application durable, verifiable, recoverable, observable, secure, and reproducible under failure.

Only after that foundation is proven should JoBot expand boards, networking, market intelligence, bulk campaigns, self-improvement, and broader agent capabilities.

Core Production Invariant (non-negotiable)

NO ACTION WITHOUT A STATE
NO STATE WITHOUT AN EVENT
NO COMPLETION WITHOUT VERIFICATION
NO SIDE EFFECT WITHOUT POLICY
NO RETRY WITHOUT IDEMPOTENCY
NO LONG RUN WITHOUT CHECKPOINT
NO MEMORY WITHOUT PROVENANCE
NO AUTONOMY WITHOUT MEASUREMENT

Sequencing Principle

- Durable execution
- State correctness
 - Browser reliability
 - Verification / evidence
 - AI reliability & evals
 - Production release
 - Capability expansion
 - High-volume autonomy

A high-throughput agent that cannot survive process death or that duplicates submissions is materially worse than a missing feature.

Part I — Current-State Assessment (Verified 2026-08-15)

1.1 Strengths (preserve and build upon)

Capability	Evidence
12-phase ASP + saga	asp/pipeline.py, asp/saga.py, DoDResult gates
8 honest adapters	VerificationResult + confirmation IDs; no fabrication

Capability	Evidence
12-provider LLM router	Cost-aware, fallback chains, daily budgets
Resume tailoring (drafter→reviewer)	A–F rubric + truthfulness gates
PDF dual-engine + ATS scoring	LaTeX + reportlab fallback
Encrypted vault + keyring	Fernet + OS keyring
Policy / caps / circuit breakers	Per-adapter isolation
Hermetic test culture	359 pytest (13 live opt-in), 18 vitest
Supply-chain hygiene	SBOM, provenance, CodeQL, Dependabot
Doctor + backup/migrate	Operational primitives already present

1.2 Maturity Scorecard

Area	Current	Target (v1.0)
Domain model	7	9
Job adapters	6	9
Application workflow	7	9.5
Durable execution	4	9.5
Task graph	3	9
State persistence	6	9
Idempotency	6	9.5
Saga / compensation	5	9
Browser automation	5	9
Security	6	9
Policy / governance	5	9
LLM routing	6.5	9
Memory	4	8.5
Observability	5	9
Evals	5	9
GUI / control plane	5	9
CI	7	9.5
Packaging / release	4	9.5
Documentation	6	9
Overall production readiness	~5	9

1.3 Critical Gaps (synthesised from all plans + live audit)

Durable Execution

- TaskGraph is in-memory; no atomic leasing, no multi-worker coordination, no persistent attempts/leases.
- No first-class event ledger or effect ledger.
- Saga compensation is closer to bookkeeping than true external-effect compensation.
- Unknown states are not first-class; ambiguous submissions can be mis-marked SUBMITTED.

Security & Vulnerabilities (plan5)

- 5 Dependabot alerts (vite high ×3, esbuild, glib residual).
- 9 CodeQL incomplete-URL-substring-sanitization findings in `infer_site()` and Workday.
- Vault keyfile: `chmod-after-write` window, no ownership/mode check on read, symlink follow.
- Tauri CSP = null; capabilities allow unrestricted args: `true`.
- Version drift across `pyproject / package.json / tauri.conf`.
- Missing governance files (SECURITY.md, CONTRIBUTING, CODE_OF_CONDUCT, FUNDING, templates).

Browser & Adapters

- Selectors hard-coded; no central registry, no healing, no drift simulation tests.
- ProxyManager and CaptchaSolver vision path unwired.
- Workday cxs-API pattern not generalised (Workable / Recuitee / Teamtailor / BambooHR opportunity).
- `form_field_memory` tier not persisted.

Release & Packaging

- No single version authority; no CHANGELOG; README 25 lines; no desktop CI / signing / auto-update.
- No coverage gate; no failure-injection / soak / GUI E2E suites.
- Publish workflow still token-based (not trusted publishing).

Other

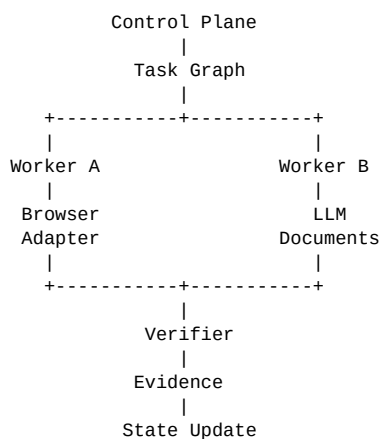
- CLI monolith (1749-line main.py).
- Silent except Exception: pass blocks.
- datetime.utcnow() deprecation.
- queues/improve.md stale relative to worklog.

Part II — Architectural Strategy

2.1 Non-Negotiable Principles (from AGENTS.md + all plans)

1. Working system > beautiful description.
2. Observable > clever.
3. Verification is a separate concern (planner/executor → verifier → reviewer/approval).
4. Local-first, privacy-preserving; BYOK for LLMs; encrypted profile vault.
5. Human-in-the-loop before consequential side effects.
6. Idempotent submissions; saga compensations; deduplication.
7. Truth-first: zero hallucinated credentials, grounded QA, PII masking.
8. Every side-effecting action carries an idempotency key and is recorded in an effect ledger.
9. Unknown is a first-class state; never blind-retry an ambiguous external effect.
10. Per-project state is file-first (markdown + structured stores for queues/events/metrics).

2.2 Target Control-Plane / Execution Separation



Agents propose; Policy decides; Execution adapter performs; Effect is recorded; Verification confirms; only then does durable state transition.

2.3 First-Class Entities to Introduce

Tasks

- Task, TaskAttempt, TaskLease, TaskEvent, TaskArtifact, TaskDependency
- Persistent statuses: PENDING → READY → CLAIMED → RUNNING → WAITING → RETRYING → VERIFYING → COMPLETED / FAILED / QUARANTINED / CANCELLED / UNKNOWN

Events

```
events (
  event_id, aggregate_type, aggregate_id, event_type, event_version,
  payload, actor, correlation_id, causation_id, created_at
)
```

External Effects

```
ExternalEffect (
  effect_id, task_id, application_id, effect_type, idempotency_key,
  request_hash, started_at, completed_at, status, external_reference,
  verification_state, compensation_state
)
```

Approvals

```
ApprovalRequest (
  id, task_id, action, risk_level, proposed_arguments, evidence,
  policy_reason, expires_at, requested_at, decided_at, decided_by,
```

```

    decision, modified_arguments
)

```

Decisions: APPROVE | EDIT | REJECT | DEFER | CANCEL

Candidate Truth

```

CandidateFact (
    fact_id, category, value, source, evidence, confidence,
    valid_from, valid_until
)

```

2.4 Risk / Trust Model

Risk	Example	Default
R0	Local read	Auto
R1	Public job scrape	Auto
R2	Resume generation	Auto
R3	Application form preparation	Auto / Draft
R4	Save application	Policy
R5	Submit application	Approval
R6	Recruiter outreach	Approval
R7	Credential modification	Approval
R8	Irreversible high-impact action	Human only

Trust is scoped per site / tool / skill, not a single global switch.

Part III — Unified Phased Roadmap

Phase	Theme	Weeks	Primary Sources	Exit Criterion
P0	Baseline Freeze & Inventory	1–2	Plan1 §46, Plan6 P0	Immutable baseline + scorecard
P1	Vulnerability Fixes & CI Hardening	2–3	plan5 W1–W10, plan4 R1	0 open Dependabot/CodeQL; security gates green
P2	Durable Execution Core	3–5	Plan1 §3–5, Plan6 P2	Kill worker mid-phase → resume correctly, no duplicate claim
P3	Application State Correctness	2–3	Plan1 §5–7,42	Interrupted applications never create duplicate external submissions
P4	Browser & Adapter Reliability	3–5	Plan1 §8, Plan3 AR-1/2, Plan6 P4	Selector healing + evidence capture + cxs family live
P5	AI Reliability & Evaluation Platform	2–4	Plan1 §10–12,22; Plan2 §5	No critical eval produces unsupported candidate claims
P6	Production Artifacts & Release Engineering	3–4	plan4 R2, plan5 W5–W6	Wheel + multi-arch Docker + desktop installers + SBOM
P7	Control Plane / GUI Completion	3–5	Plan1 §20, Plan2 §8, Plan3 F-01	Functional dashboard, approval inbox, evidence viewer
P8	Capability Expansion (P0 Features)	4–6	Plan3 C.2, Plan2 Phases 2–3	Kanban, answer bank, ATS family, follow-ups, export, apply-method classification
P9	Telemetry, Privacy & Documentation	2–3	plan4 R4–R5	Opt-in telemetry + privacy.md + docs site + governance files
P10	Release & Launch	1–2	plan4 R5	v1.0.0 tagged, artifacts on PyPI/GHCR/desktop, announcement

Total calendar estimate for v1.0.0: ~26–32 weeks of focused work (can be parallelised on independent tracks after P3).

Part IV — Detailed Work Packages

P0 — Baseline Freeze & Foundation (Weeks 1–2)

ID	Task	Verification
P0.1	Architecture inventory (every module, endpoint, dependency)	Documented
P0.2	Dependency matrix (Python / npm / Rust) + licenses	Complete
P0.3	Runtime matrix (Python 3.11/3.12/3.13; macOS/Linux/Windows/WSL2/Docker)	Documented
P0.4	Test baseline (coverage, pass rates, skip reasons)	Report
P0.5	Eval baseline	Report
P0.6	Security baseline (Dependabot, CodeQL, npm audit, pip-audit, gitleaks, bandit)	Report
P0.7	Performance baseline (key-path latency, RSS)	Report
P0.8	Production-readiness scorecard (update the table in §1.2)	Complete
P0.9	Reconcile queues/improve.md with worklog; mark wired items done	Queues truthful

Do not add product features in P0.

P1 — Vulnerability Fixes & CI/CD Hardening (Weeks 2–4)

References: plan5 entire document + plan4 R1.

ID	Task	Verification
P1.1	npm stack upgrade: vite 5.4.21 → 8.2.1, vitest → 4.x, plugin-react → 6.x, prettier, @tauri-apps/*	npm audit = 0; vitest 18/18; build OK
P1.2	Node engines ≥20.19; CI matrix 20/22	Green
P1.3	Rewrite infer_site() with urlsplit + exact netloc match; unknown → ValueError (D-1); fix Workday host check	All 9 CodeQL close; adversarial tests pass
P1.4	Document glib RUSTSEC-2024-0429 residual risk in SECURITY.md; add cargo ecosystem to Dependabot	Documented
P1.5	CI hardening: ruff defaults, pin tools, security-gates job (npm audit high, pip-audit, gitleaks, actionlint), SHA-pin actions, CodeQL +rust, trusted publishing	All jobs green
P1.6	Version unification: scripts/sync_versions.py from pyproject.toml → all package.json + tauri.conf	Drift check fails on mismatch
P1.7	Python packaging: SPDX license string, classifiers, keywords, project.urls; mypy python_version=3.11	build + twine clean
P1.8	Vault hardening: O_CREAT	O_WRONLY 0o600, ownership/mode check on read, O_NOFOLLOW
P1.9	Tauri CSP restrictive default + capability arg allowlist	tauri:dev + build clean
P1.10	Governance files: SECURITY.md, CONTRIBUTING.md, CODE_OF_CONDUCT.md, FUNDING.yml, issue/PR templates, CODEOWNERS, .editorconfig	Render on GitHub
P1.11	.gitignore additions (.venv/, .p12, .coverage., target/, etc.); move Plan1.pdf → docs/	Clean root
P1.12	Local gates scripts (gates.sh / gates.ps1)	Exit 0 on clean tree

P2 — Durable Execution Core (Weeks 4–7)

References: Plan1 §§3–5, Plan6 P2.

ID	Task	Verification
P2.1	Persistent Task / TaskAttempt / TaskLease / TaskEvent / TaskArtifact / TaskDependency tables + versioned migrations	Schema applied
P2.2	Full task state machine with atomic claim (conditional UPDATE)	Multi-worker cannot claim same task

ID	Task	Verification
P2.3	Heartbeats + lease expiry → reclaim	Kill worker → task recoverable
P2.4	Event ledger (append-only)	Audit / replay tests
P2.5	Effect ledger with idempotency_key + verification_state + compensation_state	Compensation tests
P2.6	Checkpoint after every major phase; resume from last good checkpoint	Kill mid-phase → correct resume
P2.7	Configurable retry policies (exponential + jitter; strategy change after N)	Retry tests
P2.8	Quarantine + dead-letter with evidence-rich replay	Quarantine tests

Exit criterion: Deliberately kill a worker during each major execution phase of a mock application and prove correct, non-duplicating resume.

P3 — Application State Correctness (Weeks 7–9)

ID	Task	Verification
P3.1	Formal application state machine (DISCOVERED → ... → SUBMITTED / FAILED / QUARANTINED / SUBMISSION_UNKNOWN)	Transition validation
P3.2	Every external side-effect carries idempotency key; effect ledger prevents replay	Duplicate submission rejected
P3.3	Durable ApprovalRequest entity + GUI/CLI/MCP consumers	End-to-end approval flow
P3.4	Unknown states first-class (submission_unknown, verification_unknown, ...)	Unknown → reconciliation, not blind retry
P3.5	Submission verification protocol: observe confirmation → extract ID → screenshot → reconcile portal	Ambiguous stays UNKNOWN
P3.6	Idempotency audit of all side-effecting paths	Audit report

Exit criterion: Interrupted applications never create duplicate external submissions.

P4 — Browser & Adapter Reliability (Weeks 9–12)

ID	Task	Verification
P4.1	BrowserSessionManager + BrowserPool + ProfileStore + SessionPersistence	Lifecycle tests
P4.2	Named actions (navigate, fill, submit, verify)	Action tests
P4.3	Selector registry + multi-locator fallback + healing log (Plan3 AR-2)	Drift simulation tests
P4.4	Evidence capture on every risky action (before/after screenshots, DOM snapshot, args, result, trace/application IDs)	Evidence tests
P4.5	CAPTCHA boundary + escalation path	CAPTCHA tests
P4.6	Site health / circuit breaker → quarantine → alternate source or human	Circuit tests
P4.7	Extract CxsApiAdapter base; implement Workable / Recrutee / Teamtaylor / BambooHR	Family tests; Workday unchanged
P4.8	Direct API apply paths (Greenhouse Harvest, Lever Postings, Ashby, SmartRecruiters) fully through saga	API submission tests
P4.9	Wire ProxyManager + CaptchaSolver vision path; persist form_field_memory / answer_bank	New tests

P5 — AI Reliability & Evaluation Platform (Weeks 12–15)

ID	Task	Verification
P5.1	Implement streaming for all providers; wire into CLI (--stream)	Incremental token render
P5.2	Typed LLM contracts (Pydantic structured outputs)	Schema validation
P5.3	Prompt registry + versioning (prompts/.../vN.yaml); record prompt_id/version on every call	Rollbackable

ID	Task	Verification
P5.4	Capability-aware routing + durable cost ledger / budget reservations	Cost tests
P5.5	CandidateFact system; LLM may propose but never silently mutate authoritative profile	Truthfulness tests
P5.6	Independent reviewer agent for resume/cover-letter (unsupported claims, keyword stuffing, contradictions)	Reviewer catches planted errors
P5.7	Multi-stage job matching (deterministic → cheap semantic → structured LLM → deep research)	Cost & quality metrics
P5.8	Evaluation platform as release gate: capability, reliability (pass@1/N, intervention rate), security (prompt-injection, secret exfil), long-horizon, failure-injection	Every release shows quality delta
P5.9	Prompt-injection boundary (sanitise external JD content; never execute instructions from it)	Adversarial eval suite

P6 — Production Artifacts & Release Engineering (Weeks 15–18)

ID	Task	Verification
P6.1	PyPI trusted publishing (id-token) + TestPyPI dry-run	twine check + publish succeeds
P6.2	Docker multi-arch (amd64+arm64) to GHCR + compose smoke (doctor + mock_ats)	Images pull & run
P6.3	Desktop CI (windows NSIS/MSI, macos DMG, ubuntu AppImage/deb) + cargo check	Installers launch; window title correct
P6.4	Real icon set + productName / identifier	Icons render
P6.5	tauri-plugin-updater + GitHub Releases endpoints	Dummy update applies
P6.6	Windows signing (SignPath OSS preferred) + macOS decision (defer + document Gatekeeper)	Signed artifacts or documented caveat
P6.7	CSP + capabilities already done in P1; re-verify under build	No CSP violations
P6.8	Single release.yml orchestrating all artifacts + SBOMs + provenance	One tag produces everything
P6.9	Versioned DB migrations CLI (jobot db status/migrate/backup/restore/verify)	Migration tests
P6.10	Expanded jobot doctor (runtime, security, browser, documents, AI, adapters, control plane, release) + --json	Schema tested

P7 — Control Plane / GUI Completion (Weeks 18–22)

ID	Task	Verification
P7.1	Dashboard: active work, pending approvals, failures, daily applications, costs, top matches	Live data
P7.2	Task inspector (status, owner, attempts, dependencies, phase, evidence, logs, cost)	Functional
P7.3	Application detail (job, fit, resume, cover, questions, submission state, screenshots, verification)	Functional
P7.4	Approval inbox (WHAT / WHY / RISK / EVIDENCE + Approve/Edit/Reject/Defer)	End-to-end
P7.5	Evidence / trace / cost / incident viewers	Functional
P7.6	Worker status + career funnel	Functional
P7.7	Sidecar supervision (auto-respawn, EOF/backpressure, process-tree kill, double-run lock)	Kill tests
P7.8	GUI E2E (tauri-driver) covering discover → dry-run apply → approve → dashboard	Green in CI

P8 — Capability Expansion (P0 Features for v1.0) (Weeks 20–26)

Priority order (value × fit ÷ effort):

1. **F-01** Kanban tracker + funnel analytics (applied → interviewing → offer)

2. **F-02 / F-08** Form autofill + persistent screening-answer bank (UI + CLI)
3. **F-03** Selector healing surfaced in GUI diagnostics
4. **F-04** ATS family expansion (cxs + Workable/Recruitee/...)
5. **F-10** Apply-method classification (auto / manual / email / redirect)
6. **F-06** Data export/import (CSV/JSON) round-trip
7. **F-07** Live ATS score + per-job resume variants in GUI
8. **F-05** Post-apply follow-up automation (rate-capped, opt-in, recruiter-email only)
9. **F-23** Job clipping (manual URL → tracker)

All land behind existing policy/caps and use the durable task + effect machinery.

P9 — Telemetry, Privacy & Documentation (Weeks 24–27)

ID	Task	Verification
P9.1	Opt-in Sentry (Python + JS/Rust) with aggressive redaction + kill switch <code>JOBOT_TELEMETRY=off</code>	Redaction tests; no PII in payloads
P9.2	Anonymous usage analytics (task counts, success rates, cost/run, version) — same switch	Schema tests
P9.3	<code>docs/privacy.md</code> exact match to code	Reviewed + tested
P9.4	<code>jobot</code> purge + retention defaults	Purge tests
P9.5	VitePress docs site (setup, CLI, GUI, adapters, security, telemetry, FAQ) → GitHub Pages	Builds clean
P9.6	README overhaul (badges, quickstart, architecture diagram, honest adapter status, sponsorship)	Renders + verified
P9.7	CHANGELOG (Keep a Changelog) backfilled from worklog	Traceable
P9.8	Maintainer runbook (gates, release checklist, branch policy, hotfix)	Dry-run succeeds

P10 — Release & Launch (Weeks 27–28)

ID	Task	Verification
P10.1	Community ops (stale-bot, labels, triage, public roadmap from queues)	Live
P10.2	Full RC pipeline; all gates green	Artifacts downloadable
P10.3	Tag <code>v1.0.0</code> → release → finalise CHANGELOG	Published
P10.4	Announcement (GitHub release + sponsorship call); honest live-adapter status in notes	Posted
P10.5	Post-launch queues rewrite (v1.1 roadmap)	Updated

Part V — Repository Hygiene & Cleanup

5.1 File / Directory Actions

Item	Action
<code>Plan1.pdf</code>	Move to <code>docs/</code>
Root <code>plan*.md</code> (historical)	Move under <code>docs/history/</code> after Master Plan is committed
<code>.env</code>	Confirm in <code>.gitignore</code> ; rotate any previously committed secrets
<code>queues/improve.md</code>	Full rewrite against current worklog
<code>gui/src-tauri/target/</code> , <code>__pycache__/</code> , <code>*.pyc</code> , <code>.coverage.*</code> , <code>htmlcov/</code> , <code>*.p12</code> , <code>*.pem</code> , <code>*.key</code> , <code>.venv/</code> , <code>venv/</code>	Ensure ignored
Unused / dead modules	Audit via <code>git grep</code> + <code>improve.md</code> reconciliation; archive or delete
Stale claims in README ("Patchright integration in progress")	Remove

5.2 Dependency Cleanup

- Remove unused `black` (`ruff` is the formatter).
- Document deliberate `jobspy` undeclared import (`numpy` pin + `--no-deps` recipe).
- Add `tests/test_imports.py` that imports every module under base install (guards undeclared-deps regressions).

- Add structlog, bandit, pip-audit, hypothesis (dev), opentelemetry-sdk (later), textual (TUI), fastapi (serve mode) only when their phase starts.

5.3 .gitignore Canonical Additions

```
.venv/
venv/
*.p12
*.pem
*.key
.coverage.*
htmlcov/
__pycache__/*
*.pyc
gui/src-tauri/target/
*.log
*.db-journal
.DS_Store
```

Part VI — Full Project Refactor & Performance Optimisation

6.1 Target Package Layout (incremental migration)

```
src/jobot/
■■■ core/           # events, state, tasks, workflows, errors (no external deps)
■■■ control/        # goals, approvals, budgets, trust, policies, incidents
■■■ execution/      # workers, leases, sandbox, browser, tools
■■■ ai/             # router, providers, prompts, profiles, evaluation
■■■ memory/         # semantic, episodic, procedural, retrieval
■■■ career/         # matching, scoring, market, networking, interview
■■■ applications/   # state_machine, preparation, submission, verification
■■■ adapters/       # ats/, boards/, browser/
■■■ documents/      # resume, cover_letter, pdf, ats
■■■ observability/  # tracing, metrics, events, logging
■■■ plugins/
■■■ cli/            # split: apply, scrape, resume, interview, tracker, config, admin
■■■ gui/            # sidecar bridge only
```

Migration is **incremental**: move one package at a time, keep import shims, full suite green after every move. Never a single massive directory rewrite.

6.2 Performance Targets

Area	Optimisation	Target
Import / CLI help	Lazy imports of heavy modules	jobot --help < 500 ms
LLM	Embedding cache, batch, cheaper models for classification	≥ 50 % cost reduction on matching
Browser	Pool + warm sessions + profile reuse	Cold start < 2 s
Database	Indexes, WAL checkpoint tuning, query review	Hot queries < 50 ms
Memory	Stream large responses, paginate artifacts, compaction	Steady RSS < 512 MB under normal load
Parallelism	Fan-out independent discovery / ranking tasks	≥ 2× throughput on multi-core

6.3 Code-Quality Hardening

- Split 1749-line cli/main.py into subcommand modules (main.py < 100 lines).
- Replace every except Exception: pass with explicit logging + metric.
- datetime.utcnow() → datetime.now(timezone.utc).
- Introduce structlog for structured JSON logging + correlation IDs.
- Add bandit + pip-audit to CI (already in P1).
- Coverage gate ≥ 75 % (or measured – 2 % floor, min 70 %).
- Property-based tests (hypothesis) for PII masker.
- Failure-injection suite (browser crash, network 429/500, sidecar kill, malformed forms, CAPTCHA, stale session).
- Soak test: 1 000-iteration sidecar loop (RSS bounded ±10 %, linear DB growth, zero crashes).

6.4 Sandbox & Plugin Security

Plugin install path:

Manifest → source/hash verification → declared permissions → dependency scan → sandbox capability → install → health check

Default deny. Explicit permissions for filesystem, network, browser, secrets.

Execution isolation ladder: local → restricted subprocess → container → remote sandbox.

Part VII — Documentation Generation Checklist

All of the following must exist and be accurate before the v1.0.0 tag:

Document	Location	Notes
README.md	Root	Badges, quickstart (pip / Docker / desktop), architecture diagram, honest adapter status, sponsorship
CHANGELOG.md	Root	Keep a Changelog; backfill from worklog
SECURITY.md	Root	Reporting, PGP, glib residual, telemetry pointer
CONTRIBUTING.md	Root	Full contributor runbook
CODE_OF_CONDUCT.md	Root	—
LICENSE	Root	Copyright holder line added
FUNDING.yml	.github/	—
Issue / PR templates	.github/	bug, feature, security, adapter request
CODEOWNERS	.github/	—
.editorconfig	Root	—
docs/privacy.md	docs/	Exact match to telemetry code
docs/release-policy.md	docs/	Semver + channels
docs/contracts.md	docs/	Already exists; keep current
SETUP.md	Root	Already exists; expand doctor section
VitePress site	docs/	guide/, gui/, adapters/, security/, telemetry/, faq.md → GitHub Pages
Maintainer runbook	docs/	Gates, release checklist, hotfix path

Generation order: governance files (P1) → privacy + CHANGELOG (P9) → README + docs site (P9) → final polish at tag time.

Part VIII — Decision Log (unified)

#	Decision	Default	Source
D1	infer_site unknown URLs	Raise ValueError	plan5
D2	vite upgrade	5.4 → 8.2.1 (major)	plan5
D3	glib residual	Documented accepted risk	plan5
D4	macOS notarisation	Defer v1; document Gatekeeper	plan4
D5	Windows signing	SignPath OSS; fallback document SmartScreen	plan4
D6	Auto-update host	GitHub Releases	plan4
D7	Docs generator	VitePress	plan4
D8	Coverage floor	Measured −2 %, min 70 %	plan4
D9	Crash reporting	Sentry free tier (opt-in)	plan4
D10	Browser extension	Defer post-1.0	Plan3
D11	Gmail watcher auth	Gmail API OAuth only	Plan3
D12	ToS-risk features (LinkedIn connect, bulk, proxy rotation)	Behind JOBOT_ENABLE_RISKY=1 + per-feature flags; default off	Plan3
D13	MCP mode	stdio first, SSE later	Plan3
D14	GUI for v1.0	Required (Kanban + approvals + evidence)	Consensus
D15	Autonomous final submission	Human-by-default; trusted-site policy can promote	Plan1
D16	Primary platforms	macOS + Linux Tier-1; Windows Tier-1; WSL2 + Docker supported	Consensus
D17	Patchright	Remain default behind replaceable browser interface	Consensus
D18	Local-first	Non-negotiable; cloud services optional	AGENTS.md

Part IX — Risk Register (unified)

ID	Risk	L	I	Mitigation
R1	LinkedIn / board bans Patchright	H	C	Session reuse, realistic delays, daily caps, circuit breaker, human fallback
R2	JobSpy / selector breakage	M	H	Registry + healing, health-check cron, direct-API fallback
R3	LLM rate limits / cost spikes	M	M	Fallback chain, per-provider CB, daily budget, local Ollama
R4	Duplicate external submissions	L	C	Effect ledger + idempotency + unknown states
R5	PII leakage to providers	L	C	PII masker (already present) + audit + redaction tests
R6	vite 8 / vitest 4 breakage	M	H	Build+test in P1 before proceeding; revert path
R7	GUI delays block v1	M	M	CLI remains fully functional; GUI can ship as 1.0.1 if needed
R8	Live adapters unvalidated on maintainer machine	M	M	Hermetic tests mandatory; live opt-in; release notes honest
R9	Refactor churn vs 359-test baseline	M	M	Full suite green after every AR-/P change; feature branches
R10	ToS-flagged features harm reputation	M	M	Default-off + docs + rate caps; never volume blast
R11	SQLite single-user ceiling	L	M	Documented limit; Postgres migration path in post-1.0 roadmap
R12	AGPL deters some enterprise	L	M	Clear docs; dual-licence exploration post-1.0 if demanded

Part X — Success Metrics

v1.0.0 Release Criteria

- 0 open Dependabot / CodeQL alerts
- npm audit / pip-audit / gitleaks clean
- pytest coverage $\geq 75\%$ (or agreed floor)
- ruff + mypy strict clean
- jobot doctor passes on macOS, Linux, Windows (WSL2)
- At least one verified end-to-end durable application flow (mock + one live opt-in)
- GUI dashboard + approval inbox + evidence viewer functional
- README, CHANGELOG, SECURITY, CONTRIBUTING, LICENSE present and accurate
- Artifacts on PyPI, GHCR, and desktop installers
- Privacy documentation matches code exactly
- Release notes honestly document live-adapter validation status

Post-Launch (first 30 days)

- Install-to-doctor ≤ 5 min on each Tier-1 OS
- CI green on every PR
- Telemetry opt-in $\geq 10\%$ with zero PII incidents
- ≥ 1 external contribution
- Sponsorship page live

Part XI — Post-1.0 Roadmap (v1.1+)

P1 (v1.1.x)

Gmail/IMAP watcher (OAuth), 24/7 scored digest, interview calendar, salary negotiation toolkit, session recordings in evidence viewer, MCP server mode, resume bank + diff view, local Ollama + vision captcha, multi-profile support, resume PDF ingestion, embedding-based matching, conversational jobot ask.

P2 (v1.2+)

Browser extension (separate repo), LinkedIn follow-up (ToS opt-in), stealth/proxy rotation (ToS opt-in), bulk batch-apply with hard caps (ToS opt-in), community adapter gallery, TUI (textual), HTML report generator with funnel charts, OpenTelemetry export, trust-level automation, API server mode (jobot serve), Homebrew / Scoop / Flatpak.

Architectural (v2.0+)

Event bus, plugin-ified adapters, multi-machine coordination (file-protocol hub-worker), outcome learning loop, career opportunity graph, bounded self-improvement (sandbox → eval → gate), skill extraction from successful trajectories, remote sandbox, market intelligence layer.

Part XII — Execution Protocol

1. Every work package: Plan → Implement → Verify (tests) → Gates (pytest / ruff / mypy / vitest / prettier / npm audit / sync_versions) → worklog + queues + CHANGELOG entry.
2. Feature branches + PRs; one-change eval loops; no giant prompt surgery.
3. Full suite green after every phase.
4. Live adapters remain opt-in; release notes stay honest.
5. File-based state (queues, worklog, decisions.md) updated continuously, not only at the end.
6. When in doubt, prefer the narrower, verified, recoverable path over the broader, clever, unverified path.

Clarification Questions (must be answered before P2 begins)

1. Is the primary target still personal/local-first tool with open-source distribution, or is a future hosted SaaS path required in the architecture now?
2. Final autonomy policy for submission: always human, human-by-default + trusted-site promotion, or fully autonomous under policy?
3. Geographic priority ordering for adapters (India vs US/EU) for the first three months after v1.0?
4. Hard deadline (job-search timing) or continuous delivery?
5. Acceptable residual risk level for glib / Patchright / board ToS?

This Master Plan is the single authoritative roadmap. All prior plans (Plan1–Plan6, plan.md, historical unified plans) are superseded for execution purposes; they remain in docs/history/ for provenance.

The long-term moat is not the number of job boards. It is the combination of:

durable execution + trustworthy candidate data + outcome learning + evidence + human-governed autonomy + career intelligence.

That is what turns JoBot from an automation script into a genuine career operating system.

End of Master Plan

Generated 2026-08-15 from exhaustive synthesis of all source plans, live repository audit, and AGENTS.md doctrine.